الملف الرابع: محرك Sigma (sigma_engine_go) — عقل الكشف الأمني

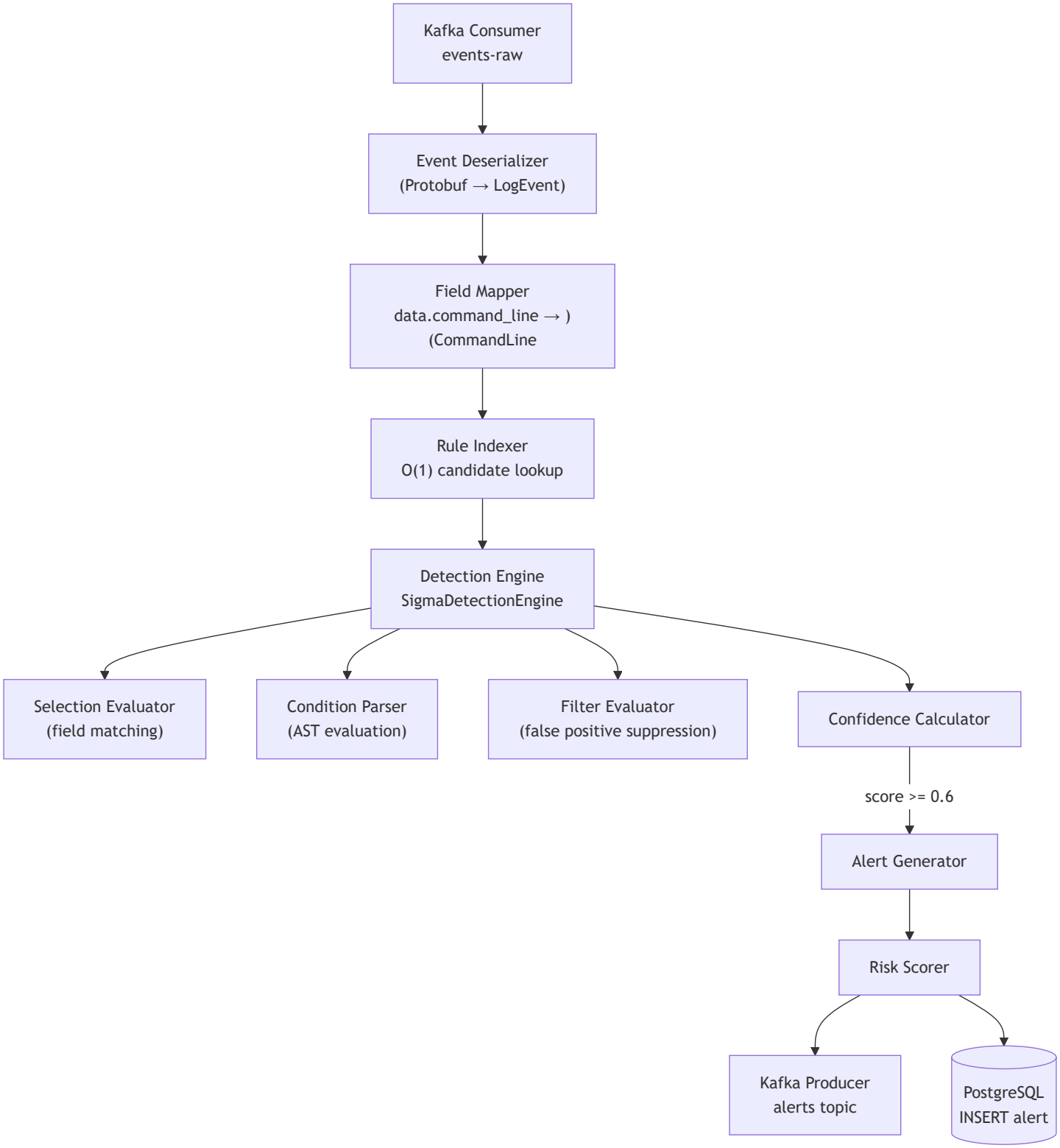
أولاً: ما هو Sigma ولماذا هو معيار الكشف الأمني؟

Sigma هو معيار مفتوح المصدر لكتابة قواعد الكشف الأمني. فكرته شبيهة بفكرة قواعد Snort لكشف هجمات الشبكة، لكنه مُصمَّم لأحداث السجلات (Log Events). أنشأه مجتمع أمن المعلومات ليكون لغة مشتركة لوصف الأنماط الأمنية المشبوهة بطريقة يمكن تحويلها لأي نظام SIEM.

قاعدة Sigma تُكتَب بصيغة YAML وتصف: ما نوع الحدث المستهدَف (category: process_creation، product: windows)، وما الشروط التي يجب تحقّقها (مثلاً: CommandLine يحتوي على "powershell" و"-enc" معاً)، ومستوى الخطورة، والتعيين إلى MITRE ATT&CK.

مشروع **SigmaHQ** يحتفظ بآلاف القواعد الجاهزة التي تُغطي معظم أنماط الهجمات المعروفة. هذا المشروع يستخدم هذه القواعد ويضيف عليها قواعد مخصصة.

ثانياً: بنية محرك Sigma من الداخل



ثالثاً: كيف تُحمَّل قواعد Sigma وتُفهرَس؟

عند بدء التشغيل، يقرأ المحرك جميع ملفات `.yml` من مجلد `/sigma_rules` . لكل ملف يُحوَّل من YAML إلى هيكل بيانات Go يُسمى `domain.SigmaRule` الذي يحتوي على:

- معرف فريد (ID)، الاسم والوصف.
- `logsource` : يُحدد نوع الحدث المستهدَف (category, product, service).
- `detection` : قاموس من الـ Selections وشرط منطقي (condition).
- `level` : مستوى الخطورة (informational / low / medium / high / critical).
- `tags` : يتضمن تعيينات MITRE ATT&CK مثل `attack.t1055` .
- `status` : حالة القاعدة (stable / test / experimental).

بعد تحميل القواعد، يبني المحرك **Rule Indexer** — وهو فهرس ثلاثي الأبعاد (product, category, service). هذا يعني أنه عند استلام حدث بـ `product=windows, category=process_creation` ، لا يتحقق المحرك من كل القواعد — بل يجلب فقط القواعد المُعرَّفة لهذا النوع من الأحداث في O(1) (وقت ثابت). هذا تحسين جذري في الأداء يُقلّل القواعد المُقيَّمة لكل حدث من الآلاف إلى عشرات أو مئات.

رابعاً: آلية الكشف خطوة بخطوة

عندما يصل حدث من Kafka، تمر المعالجة بخمس خطوات متسلسلة:

الخطوة 1: تعيين الحقول (Field Mapping)

الأحداث القادمة من الوكيل تستخدم مخطط حقول خاص بالمشروع (مثل `data.executable` ، `data.command_line`). قواعد Sigma المعيارية تستخدم اصطلاحات مختلفة (`Image` ، `CommandLine`). الـ FieldMapper يُحوّل بين الاثنين. مثلاً:

- `data.command_line` → `CommandLine`
- `data.executable` → `Image`
- `data.parent_executable` → `ParentImage`
- `data.name` → `Image` (للعمليات)

الخطوة 2: الكشف عن القواعد المرشّحة

`Rule Indexer` يُعيد قائمة بالقواعد المطابقة لـ `(product, category, service)` الخاص بالحدث. هذه القائمة هي الـ "candidates".

الخطوة 3: تقييم كل قاعدة مرشّحة

لكل قاعدة يُقيّمها المحرك على ثلاث مراحل:

أولاً — تقييم الـ Selections: كل قاعدة تحتوي على واحد أو أكثر من الـ Selections. كل Selection هو مجموعة شروط AND: جميع الحقول داخله يجب أن تتحقق معاً. مثلاً:

```
:selection_main
:CommandLine|contains|all
'powershell' -
'enc-' -
'nop-' -
```

هذا يعني "CommandLine يحتوي على 'powershell' AND '-enc' AND '-nop' كلها معاً".

يدعم المحرك عشرات الـ modifiers: `contains` ، `startswith` ، `endswith` ، `equals` ، `re` (regex)، `base64offset` ، `all` ، `cidr` ، `windash` ، وغيرها. كل modifier له منطق تطابق مختلف يُطبَّق على قيمة الحقل.

ثانياً — تقييم الـ Condition: الـ Condition يحدد كيف تُجمَّع نتائج الـ Selections. مثلاً:

- `selection_main` → يجب أن يتحقق selection_main.
- `1 of selection_*` → يكفي تحقق أي selection يبدأ اسمه بـ selection_.
- `all of them` → يجب تحقق جميع الـ selections.
- `selection_main and not filter_*` → تحقق main مع عدم تحقق أي filter.

يُحوِّل المحرك نص الـ Condition إلى AST (Abstract Syntax Tree) ثم يُقيّمه بناءً على نتائج الـ Selections.

ثالثاً — تقييم الـ Filters (منع False Positives): إذا وُجد selections يبدأ اسمها بـ `filter` ، وتحقق أي منها، تُلغَى نتيجة الكشف. هذا للحالات التي يكون فيها السلوك مطابقاً للقاعدة لكن في سياق معروف بأنه بريء (مثل نشاط أدوات إدارية شرعية).

الخطوة 4: حساب درجة الثقة (Confidence Score)

درجة الثقة تُعبِّر عن مدى اليقين من أن الحدث يمثّل تهديداً حقيقياً. تُحسَب بالمعادلة:

```
confidence = base_confidence × field_factor × context_factor
```

- **base_confidence** يأتي من مستوى القاعدة: critical=0.95، high=0.85، medium=0.65، low=0.45، informational=0.25.
- **field_factor** = (الحقول التي تطابقت) / (إجمالي الحقول في الـ selections المتطابقة). كلما تطابقت حقول أكثر، ارتفعت الثقة.

- **context_factor** اختياري: إذا كانت القاعدة تستخدم `ParentImage` لكن الحدث لا يحتوي عليه، تُخفَض الثقة بنسبة 20%.

أي نتيجة بـ confidence أقل من 0.6 (قابل للتخصيص) تُلغَى ولا تُولِّد تنبيهاً. هذا يُقلّل False Positives بشكل ملحوظ.

الخطوة 5: التجميع للحد من "تعب التنبيهات"

بدلاً من توليد تنبيه منفصل لكل قاعدة تطابقت، يستخدم المحرك `DetectAggregated()` التي تجمع كل نتائج التطابق لحدث واحد في تنبيه واحد. مثلاً إذا تطابق حدث واحد مع 5 قواعد، يُولَّد تنبيه واحد يتضمن جميع القواعد الخمس — بدلاً من خمسة تنبيهات منفصلة تُربك المحلل.

خامساً: حساب درجة المخاطرة (Risk Score)

بعد توليد التنبيه، يُطبَّق عليه Risk Scorer يُعطيه درجة مخاطرة من 0 إلى 100. الدرجة تعتمد على:

- خطورة الحدث (severity).
- حساسية الجهاز (criticality: low/medium/high/critical).
- البيئة (production أعلى من development).
- درجة الثقة في الكشف.
- عدد مؤشرات MITRE ATT&CK المتطابقة.

هذا يعني أن نفس التنبيه على جهاز خادم إنتاج حساس (critical) يحصل على درجة مخاطرة أعلى بكثير من نفس التنبيه على جهاز اختبار.

سادساً: WebSocket للتحديثات الفورية

المحرك يحتوي على WebSocket Server يسمح للوحة التحكم بالاستماع للتنبيهات الجديدة في الوقت الفعلي. عندما يُولَّد تنبيه جديد، يُرسَله المحرك لجميع المتصلين عبر WebSocket فوراً دون حاجة للوحة التحكم لإجراء polling.

سابعاً: قواعد Sigma المُستخدَمة في المشروع

مجلد `/sigma_rules` يحتوي على عدة فئات من القواعد:

`/rules` : القواعد الأساسية المُجرَّبة والمستقرة. تشمل: كشف PowerShell مُشفَّر، تنفيذ أوامر WMI، إنشاء خدمات Windows مشبوهة، نشاط Registry للـ persistence، حقن العمليات، تفريغ لسائل LSASS.

/rules-threat-hunting : قواعد لصيد التهديدات. أقل حساسية في الكشف لكنها تُعطي إشارات مبكرة عن نشاط محتمل.

/rules-emerging-threats : قواعد لتهديدات جديدة ناشئة. درجة تجريبية أعلى.

/post_isolation : قواعد مُفعَّلة خصيصاً بعد عزل الجهاز — تُغطي سيناريوهات التحقيق الجنائي بعد الاختراق.

/rules-compliance : قواعد الامتثال التنظيمي (مثل كشف تغييرات سياسة كلمات المرور).

ثامناً: نظام ارتباط الأحداث (Correlation Engine)

بجانب محرك الكشف المباشر، يحتوي المحرك على نظام ارتباط أكثر تعقيداً يتتبع تسلسل الأحداث عبر الزمن. مثلاً: إذا أنشأت عملية معينة اتصالاً خارجياً خلال 60 ثانية من تحميلها لملف DLL غير موقَّع — فهذا نمط هجوم أكثر تقدماً يتطلب ربط أحداث متعددة.

يحتفظ `CorrelationManager` بـ "lineage cache" في Redis يتتبع سلاسل الأحداث المرتبطة بنفس العملية أو نفس المستخدم خلال نافذة زمنية محددة. هذا يسمح بكشف هجمات متعددة المراحل لن يكتشفها الكشف المباشر على حدث واحد.

تاسعاً: واجهة REST API لمحرك Sigma

يُوفّر المحرك REST API مؤمَّناً بـ JWT (نفس رموز Connection Manager) يشمل:

- `GET /api/v1/sigma/rules` — استرجاع قائمة قواعد Sigma مع خيارات فلترة.
- `POST /api/v1/sigma/rules` — إضافة قاعدة Sigma مخصصة.
- `PUT /api/v1/sigma/rules/{id}` — تعديل قاعدة.
- `DELETE /api/v1/sigma/rules/{id}` — حذف قاعدة.
- `GET /api/v1/sigma/alerts` — استرجاع التنبيهات مع فلترة حسب الخطورة والجهاز والحالة.
- `PATCH /api/v1/sigma/alerts/{id}` — تحديث حالة تنبيه (مثلاً إغلاقه أو تصنيفه كـ false positive).
- `GET /api/v1/sigma/stats` — إحصائيات المحرك (عدد الأحداث المعالجة، القواعد النشطة، زمن المعالجة).
- `GET /api/v1/sigma/alerts/stream` — WebSocket للتنبيهات الفورية.